PBL Milestone 1 (Revision 2): Revised Algorithm Sections

Project LIBRAS: Library Borrowing and Records Administration System

Algorithms and Complexity (ALGOCOM) and Software Engineering (SOFTENG)

University of Makati, College of Computer Science, Section III-BCSAD, Group 1, Academic Year 2026-2027

Prototype benchmark code and raw results: `github.com/necesario0/LIBRAS`, directory `benchmarks/`.

Revision Summary

This revision replaces the algorithm choices of the first submission with the algorithms that performed best in a measured comparison against alternatives. Only sections that changed are included here. Section numbers match the original milestone so each revised section can replace its original.

Original section	What changed	Reason
2.1 Strategy schema	New names, new baselines, evidence-based justification	Algorithms were compared empirically (Section 3.5)
2.2 Invariants	Added the pruning lemma and the queue-order invariant	Proves the pruning rule is safe
2.3 Pseudocode	Algorithm 1 uses a flat array trie; Algorithm 2 is an indexed priority query	Smaller, faster trie; durable, race-free queue
3.1 to 3.4 Complexity	Corrected visited-node bound; measured averages; new space figures	Original node-count bound was one factor of m too low
3.5 (new)	Empirical algorithm selection	Shows why these algorithms were chosen
4.1 Architecture	Search runs as a module in the Node.js API; no separate Python service or Redis	Matches the project technology stack
5.2 Tickets	Rewritten for TypeScript and PostgreSQL	Matches the technology stack
Task 1, 6 and poster	Statements that no longer hold are listed with replacements	Consistency (last section)

Final algorithm names

Role	Name	Original name
Catalog search	**Bounded Levenshtein Trie Search**	Bounded Levenshtein Trie Automaton Search
Hold dispatch	**Role-Weighted Priority Queue Dispatcher**	Role-Weighted Composite Min-Heap Hold Dispatcher

Task 2: Algorithmic Paradigm Selection and Logic Specification (Revised)

2.1 Strategy Specification Schema

The architecture keeps the dual-engine design. Catalog search uses dynamic programming and branch-and-bound over a trie. Hold dispatch uses a greedy priority queue, ordered by an index in PostgreSQL.

Schema field	Specification
Algorithm names	(1) Bounded Levenshtein Trie Search; (2) Role-Weighted Priority Queue Dispatcher
Design paradigms	Dynamic programming (one memoized DP row per trie node); branch and bound (prune a subtree when every cell of its DP row exceeds the radius); greedy choice (serve the minimum-score hold)
Foundational references	Levenshtein (1966); Ukkonen (1985); Cormen, Leiserson, Rivest and Stein (CLRS, 4th ed., 2022); Knuth (1998); Schulz and Mihov (2002); PostgreSQL 16 documentation
Baselines compared (search)	SQL `ILIKE '%term%'`; PostgreSQL `levenshtein_less_equal` scan; PostgreSQL `pg_trgm` GIN index; linear scan with cutoff DP; BK-tree; SymSpell; bigram inverted index; trie with per-node objects
Baselines compared (queue)	Unsorted scan; sorted array; binary heap (with and without an id index); 4-ary heap; binary heap with lazy deletion; pairing heap; PostgreSQL indexed query
Computational justification	The flat trie answered 50,000-token queries at k = 2 with p95 latency 0.73 ms in 4.4 MB, the fastest exact option within the memory limit (Section 3.5). The PostgreSQL queue dispatched in 1.7 ms against a 10 ms limit, in the same order as a binary heap, and served each hold once under 50 concurrent clients.
Invariants	Pruning lemma (Section 2.2); queue-order invariant (Section 2.2)

2.2 Mathematical Invariants and Recurrence Relations

Recurrence (edit distance row). Let `Q = q_1 ... q_m` be the query and let `u` be the string spelled by a trie node, whose last letter is `a`. Let `R_u` be the DP row of `u`, with cells `R_u[0..m]`, where `R_u[c]` is the Levenshtein distance between `u` and the prefix `q_1 ... q_c`. For a child node `v` of `u` reached by letter `a`:

```
R_v[0] = R_u[0] + 1
R_v[c] = min( R_u[c-1] + (0 if q_c = a else 1),   // match or substitute
              R_u[c]   + 1,                       // delete a from the word
              R_v[c-1] + 1 )                      // insert q_c into the word
```

The row of the root is `R_root[c] = c`. The distance between the query and the word spelled at a node is the last cell, `R_v[m]`.

Lemma 1 (monotone row minimum). For every child `v` of `u`, `min(R_v) >= min(R_u)`.

Proof. By induction on `c`. `R_v[0] = R_u[0] + 1 >= min(R_u)`. Assume `R_v[c-1] >= min(R_u)`. Each of the three terms defining `R_v[c]` is at least `min(R_u)`: the first two are `R_u` cells plus a non-negative amount, and the third is `R_v[c-1] + 1`. Hence `R_v[c] >= min(R_u)`. QED.

Lemma 2 (pruning is safe). If `min(R_u) > k`, then every word in the subtree of `u` has edit distance greater than `k` from the query.

Proof. By Lemma 1, the row minimum never decreases along a path from `u` to a descendant `w`, so `min(R_w) > k`. In particular the last cell satisfies `R_w[m] >= min(R_w) > k`. QED.

Consequence: the search returns exactly the words within distance `k`, with no false negatives and no false positives. This was also verified by 27 automated tests and by zero mismatches against a brute-force reference in every benchmark query.

Observation (no pruning at shallow depth). At depth `d`, `R[0] = d`, so `min(R) <= d`. Pruning therefore cannot occur at depth `d <= k`. For `k = 2`, every trie node at depth 1 or 2 is always expanded.

Queue-order invariant. Each hold `r` has a score `PriorityScore(r) = t_r - W(role_r)`, where the role weights `W` are as defined in Section 1.2.2 (Faculty 86,400,000 ms; NTP 43,200,000 ms; Student 0; Visitor -86,400,000 ms). Ties are broken by arrival sequence `seq`. The queue is ordered by the key `(score, seq)`, and dispatch always removes the entry with the smallest key. Because `seq` is unique, the order is total and the winner is unique.

2.3 Structured Algorithmic Pseudocode

Algorithm 1: Bounded Levenshtein Trie Search (flat representation)

The trie is built once from the lexicon and is immutable. Nodes are numbered in breadth-first order, so the children of node `v` are the edges in the range `child_start[v]` to `child_start[v+1] - 1`.

```
structure FlatTrie:
    child_start : Int32[T + 1]   // T = number of trie nodes
    edge_char   : Uint16[E]      // letter on each edge (E = T - 1)
    edge_target : Int32[E]       // child node of each edge
    word_of     : String[T]      // word that ends at node v, or null
    max_depth   : Integer        // length of the longest word

function Search(trie, query, k) -> List<Match>:
    m = length(query)
    rows = Int32[(max_depth + 2) * (m + 1)]   // rows[d] = DP row, depth d
    for c in 0..m: rows[0][c] = c             // root row
    results = []
    if word_of[0] != null and rows[0][m] <= k:
        add (word_of[0], rows[0][m]) to results
    stack = [ (node = 0, next_edge = child_start[0]) ]
    while stack is not empty:
        frame = top(stack)
        if frame.next_edge == child_start[frame.node + 1]:
            pop(stack); continue
        e = frame.next_edge;  frame.next_edge = e + 1
        u = edge_target[e];   letter = edge_char[e]
        d = size(stack)                       // depth of the child u
        rows[d][0] = rows[d-1][0] + 1
        for c in 1..m:
            sub = rows[d-1][c-1] + (query[c-1] == letter ? 0 : 1)
            rows[d][c] = min(sub, rows[d-1][c] + 1, rows[d][c-1] + 1)
        if word_of[u] != null and rows[d][m] <= k:
            add (word_of[u], rows[d][m]) to results
        // BRANCH AND BOUND: expand u only if some cell is within k
        if min(rows[d]) <= k and u has children:
            push(stack, (node = u, next_edge = child_start[u]))
    sort results by (distance ascending, relevance score descending)
    return results
```

The recursive version in the first submission is equivalent. The flat version replaces one object per node with three typed arrays, one preallocated block of DP rows, and an explicit stack in place of recursion.

Algorithm 2: Role-Weighted Priority Queue Dispatcher (PostgreSQL)

The queue is a table with an ordered index. The index is a B-tree, so finding and removing the minimum are logarithmic, as in a heap.

```
CREATE TABLE holds (
    id     BIGINT  PRIMARY KEY,   -- plus patron, copy, status columns
    title  INTEGER NOT NULL,      -- one queue per title
    score  BIGINT  NOT NULL,      -- arrival_ms minus role weight (ms)
    seq    BIGINT  NOT NULL       -- arrival order, breaks ties
);
CREATE INDEX holds_queue ON holds (title, score, seq);

-- Enqueue                                                    O(log K)
INSERT INTO holds (id, title, score, seq) VALUES ($1, $2, $3, $4);

-- Dispatch (extract minimum)                                 O(log K)
DELETE FROM holds
WHERE id = (SELECT id FROM holds
            WHERE title = $1
            ORDER BY score, seq
            LIMIT 1
            FOR UPDATE SKIP LOCKED)
RETURNING id;

-- Cancel or expire a hold                                    O(log K)
DELETE FROM holds WHERE id = $1;
```

```
function CalculateScore(arrival_ms, role) -> Int64:
    weights = { Faculty: 86400000, NTP: 43200000,
                Student: 0, Visitor: -86400000 }
    return arrival_ms - weights.get(role, weights.Visitor)

function DispatchReturnedCopy(title, copy_id):       // one transaction
    BEGIN
    winner = Dispatch(title)                         // statement above
    if winner is null:
        COMMIT; return NO_ACTIVE_HOLDS
    UPDATE book_copies SET status = 'READY_FOR_PICKUP' WHERE id = copy_id
    COMMIT
    return winner
```

`FOR UPDATE SKIP LOCKED` locks the chosen row for the duration of the transaction. A concurrent dispatch that finds the top row locked takes the next row instead of waiting, so two desks never receive the same hold. The in-memory equivalent of this queue is the array-backed binary min-heap of the first submission (`SiftUp`, `SiftDown`, `ExtractMin`), which remains a correct reference implementation and was used to verify the order of the PostgreSQL queue.

Task 3: Theoretical Asymptotic Analysis and Scalability Profiling (Revised)

Notation: `m` = query length; `k` = edit radius (2); `|Σ|` = alphabet size; `T` = number of trie nodes; `N` = number of lexicon words; `L_avg` = average word length; `K` = number of holds in a queue.

3.1 Worst-Case Time Complexity (O)

Algorithm 1. Computing one DP row costs `O(m)`. The search visits a node only if the row minimum of its parent is at most `k` (Lemma 2 and the pseudocode). Because `R_u[c]` is the edit distance from `u` to the prefix `q_1 ... q_c`, a parent has row minimum at most `k` exactly when its string is within `k` edits of some prefix `q_1 ... q_j` of the query.

- There are `m + 1` prefixes.

- The Levenshtein ball of radius `k` around a string of length at most `m + k` contains `O((m |Σ|)^k)` strings, because each of at most `k` edits chooses a position (`O(m)` choices) and an operation with a letter (`O(|Σ|)` choices).
- So the number of expanded nodes is at most `O(m^(k+1) |Σ|^k)`, and each expanded node has at most `|Σ|` children, each of which is visited.

```
Nodes visited      V = O( m^(k+1) * |Σ|^(k+1) )
For k = 2:         V = O( m^3 * |Σ|^3 )
Time, full rows:   O( m * V ) = O( m^4 * |Σ|^3 )
Time, banded rows: O( k * V ) = O( m^3 * |Σ|^3 )     (see note)
```

Note: a DP row only needs the cells within `k` of the diagonal (Ukkonen, 1985), which reduces the per-node cost from `O(m)` to `O(k)`. The implemented and benchmarked version computes full rows.

Correction to the first submission. The first submission bounded the visited nodes by the single ball around the whole query, `O(m^2 |Σ|^2)`, and the time by `O(m^3 |Σ|^2)`. That omits the balls around the `m` shorter prefixes, which add one factor of `m`. The corrected growth was confirmed experimentally on an exhaustive dictionary (every string up to length `m + k + 1`), the case where no branch is absent. For `k = 2` the ratio of visited nodes to `m^3` settles to a constant while the ratio to `m^2` keeps growing:

Alphabet	m = 6	m = 10	m = 14	visited / m^3 at largest m	visited / (m^3 |Σ|^3)
2 letters	388	1,492	3,876	1.4	0.17
3 letters	2,409	10,767	n/a	10.9 (m = 12)	0.40
4 letters	7,060	31,172	85,428	31.1	0.49

(Cells are visited-node counts. The largest m tested was 22 for 2 letters, 12 for 3 letters and 14 for 4 letters. Source: `benchmarks/results/worst-case.md`.)

Dependence on catalog size. Both `V` and the running time are bounded independently of `N`. However, the visited nodes can never exceed the trie size `T`, and for realistic parameters (`m = 12`, `|Σ| = 26`) the ceiling `m^3 |Σ|^3` far exceeds `T`, so the ceiling does not limit the cost in practice. The actual cost depends on how many trie nodes fall inside the edit region, which grows with `N` (Section 3.2).

Algorithm 2.

Operation	Time	Note
Enqueue (`INSERT`)	`O(log K)`	B-tree insertion
Dispatch (`DELETE ... LIMIT 1`)	`O(log K)`	Descend to the first index entry, delete it
Peek (`SELECT ... LIMIT 1`)	`O(log K)`	An in-memory heap is `O(1)`
Cancel or expire	`O(log K)`	Primary-key lookup, then index maintenance

3.2 Average-Case and Best-Case Time

Best case (Algorithm 1). By the observation in Section 2.2, no node of depth `d <= k` is ever pruned, so the search always visits every trie node of depth at most `k`, plus at least one root-to-leaf path of length `m` when the query is an indexed word. The lower bound is `Ω(m + n_k)`, where `n_k` is the number of trie nodes of depth at most `k` (at most `1 + |Σ| + |Σ|^2` for `k = 2`). The first submission's `Ω(m)` bound assumed pruning at depth 1 or 2, which cannot occur.

Average case (Algorithm 1), measured. The average case depends on the real character distribution of the catalog. On the synthetic 50,000-token catalog, with the literal trie implementation and queries of length 12:

Edit radius k	Nodes visited per query	Trie nodes never visited	Matches per query
1	1,036	99.6%	0.4
2	8,401	96.7%	0.9

The first submission estimated about 40 node visits at `k = 2`, and 98.4% of branches pruned. The measured pruning is 96.7% at `k = 2` and 99.6% at `k = 1`, so pruning is effective but the visited count is about 200 times larger than estimated. The estimate `(m (b_eff)^k)` assumed a branching factor that is not defined for the pruned region.

The number of visited nodes grows sublinearly with catalog size. With `m = 12` and `k = 2`:

Tokens in catalog	5,000	20,000	50,000	100,000	200,000
Nodes visited	2,708	5,517	8,152	11,613	15,083

Between 5,000 and 200,000 tokens (a factor of 40), visited nodes grew by a factor of 5.6, roughly proportional to `N^0.47` on this data. This is an empirical observation on synthetic data, not a proved bound. It replaces the first submission's statement that cost is independent of `N`.

Algorithm 2. For the in-memory reference heap, a random insertion sifts up 0.58 levels on average under a realistic role mix (70% Student, 15% Faculty, 10% NTP, 5% Visitor) and 0 levels when all patrons have the same role, because arrival times increase. This replaces the first submission's estimate of 1.6 levels. For the PostgreSQL queue, insertion and dispatch are `O(log K)` index operations regardless of the arrival pattern.

3.3 Auxiliary Space Complexity (O)

Algorithm 1. The flat trie stores `T` nodes, where `T <= N * L_avg`.

```
Space = O( T )  words
Measured: T = 0.545 * N * L_avg (synthetic catalog)
Measured memory: about 17 bytes per trie node
   50,000 tokens ->    255,689 nodes ->  4.4 MB
  200,000 tokens ->  about 1 million  -> 16.9 MB
  400,000 tokens ->                      32.6 MB
```

The working memory of one query is the DP row block, `(L_max + 2)(m + 1)` 32-bit integers, which is about 17 KB for `L_max = m = 64`. The same lexicon stored as one JavaScript object per node (the literal pseudocode) needed 55 MB at 50,000 tokens, about 12 times more.

Algorithm 2. The queue is stored in PostgreSQL as `O(K)` rows plus an `O(K)` index. It uses no memory in the Node.js process.

3.4 Big Data Dimensions (Volume, Velocity, Variety)

Dimension	Parameters	Impact and mitigation
Volume	50,000+ tokens; 5,000 holds per queue	The flat trie occupies 4.4 MB at 50,000 tokens and 16.9 MB at 200,000 tokens, so memory is not a constraint. The hold queue lives in PostgreSQL, not in application memory.

Dimension	Parameters	Impact and mitigation
Velocity	250 searches/s peak; 50 concurrent return events	One Node.js thread answered 937 searches/s (k = 2, 50,000 tokens, in-process), about 3.7 times the peak load. Fifty concurrent clients drained 2,000 holds in 0.42 s through PostgreSQL.
Variety	ISBN-10, ISBN-13, Dewey call numbers, MARC21 titles	Titles, authors, and call numbers and ISBNs each get their own trie so the alphabet stays small. On a 20,000-token call-number lexicon the flat trie answered k = 2 queries with p95 latency 1.25 ms in 1.0 MB. Text is normalized (NFKC, lowercase) at ingestion.

3.5 Empirical Algorithm Selection (New)

Purpose. The two algorithms were chosen by comparing them against alternatives on the same data, queries, and machine. The team's requirements were: for search, the lowest latency that does not consume much more memory than the other options; for the hold queue, the option that works best while staying light on the device.

Method. All numbers come from the prototype in `benchmarks/` (TypeScript, Node.js 24, PostgreSQL 16 in a local Docker container, AMD Ryzen 7 PRO 5850U laptop with 15.3 GB RAM). The real UMak catalog was not available, so a seeded synthetic catalog of 50,000 unique tokens (mean length 9.4) was generated, with queries made by applying 0 to 2 random edits to catalog words plus 5% random non-matching strings. Every search candidate was checked against a brute-force Levenshtein reference and returned zero mismatches. Latency is per query after warm-up. Memory is bytes retained after a forced garbage collection.

Selection rule for search. A candidate must (1) return exact results, (2) sustain at least 250 queries/s on one thread at k = 2, and (3) retain no more than twice the memory of the smallest candidate that passes (1) and (2). Among those, the lowest p95 latency wins.

Search results (50,000 tokens, k = 2):

Candidate	p95 (ms)	Queries/s	Memory	Outcome
SymSpell	0.19	15,139	318 MB	Fastest; fails memory rule
Trie, one object per node	2.93	553	55 MB	Fails memory rule
BK-tree	26.2	55	11 MB	Fails throughput and memory rules
Linear scan, cutoff DP	13.5	113	none	Fails throughput rule
Bigram inverted index	1.97	823	5.6 MB	Passes
Flat trie (chosen)	**1.93**	**937**	**4.4 MB**	**Passes; fastest passing**

The bigram index and the flat trie were almost tied in that single run, so they were compared again with five repeated runs in alternating order and catalog sizes up to 400,000 tokens (p50 / p95 in ms, k = 2):

Tokens	Flat trie	Bigram index	Memory, trie / bigram
20,000	0.27 / 0.51	0.23 / 0.40	1.8 / 2.2 MB
50,000	0.42 / 0.73	0.69 / 1.25	4.4 / 5.6 MB
100,000	0.62 / 1.13	1.97 / 4.25	8.7 / 11.1 MB
200,000	0.84 / 1.53	4.48 / 8.05	16.9 / 22.6 MB
400,000	1.32 / 2.44	13.0 / 21.6	32.6 / 46.3 MB

The flat trie is faster from 50,000 tokens upward and smaller at every size. Its advantage grows with catalog size.

PostgreSQL search options (50,000 tokens, k = 2, client round trip including Docker port forwarding):

Candidate	p95 (ms)	Matches found	p95 with 50 clients (ms)
`ILIKE '%term%'` (first submission's baseline)	23.9	3.7%	312
`levenshtein_less_equal` scan	10.5	100%	186
`pg_trgm` GIN index, default threshold	2.6	16.9%	85
`pg_trgm` GIN index, threshold 0.1	12.5	69.8%	128

`ILIKE` is a substring match, not a fuzzy match, so it finds almost none of the typo matches. `pg_trgm` is fast but returns only 17% to 70% of the words within two edits, because trigram similarity does not guarantee edit distance. Neither can replace an exact fuzzy search. The exact scan is correct and meets the 1.0 s limit but is slower than the flat trie and competes with transactions for database CPU.

Hold queue results (5,000 holds, role mix 70/15/10/5):

Candidate	Steady state (ns/op)	With cancellations (ns/op)	Cancel cost
Binary heap, no id index	89	123	`O(K)`
Binary heap with id index	217	102	`O(log K)`
4-ary heap with id index	169	89	`O(log K)`
Binary heap, lazy deletion	101	76	`O(1)`
Pairing heap, lazy deletion	90	82	`O(1)`
Sorted array	208	133	`O(K)`
Unsorted scan	6,568	588	`O(K)`
PostgreSQL indexed query (chosen)	**1,670,000 (p50)**	**1,620,000 (p50)**	**`O(log K)`**

Every in-memory option is about 50,000 times faster than the 10 ms limit, so speed does not separate them. The PostgreSQL query is slower (p99 2.58 ms) but still four times inside the limit. It was chosen because the queue must be correct and durable. It served holds in exactly the same order as the in-memory heap for 500 consecutive dispatches. With 50 concurrent clients draining 2,000 holds, every hold was served exactly once. It survives a server restart, commits atomically with the loan record, and uses no memory in the application process. Under concurrency `SKIP LOCKED` does not guarantee strict global order, because a client skips a row that another client holds locked, but no hold is lost or served twice.

Limitations. The catalog is synthetic, so branching and pruning figures must be re-checked on real holdings. All numbers come from one laptop, and PostgreSQL ran in a Docker container, so round trips include port-forwarding overhead. In-process search was measured on a single thread and was not driven through the Express server with 50 simulated users. Search matches whole tokens only; the relevance score, multi-token titles and Unicode normalization were not benchmarked.

Task 4: Decoupled Software Architecture (Revised Section 4.1)

4.1 System Component Architecture and Boundary Isolation

The principle of the first submission stands: heavy algorithmic work stays out of controllers and database mapping code. It is implemented as module boundaries inside the Node.js API instead of a separate service. The tiers are:

1. **Presentation tier.** A React Native (TypeScript) mobile app for students, faculty and non-teaching personnel to search the catalog, reserve books and view loans, and a React.js (TypeScript) web portal for librarians.
2. **Hosting and ingress.** Render hosts the API and the web portal with HTTPS termination and continuous deployment from GitHub. JWT authentication and rate limiting run as Express middleware.
3. **Business and persistence tier.** A Node.js, Express and TypeScript API handles business logic and circulation transactions. PostgreSQL stores catalog metadata, accessions, borrower accounts, loan history, fines and the hold queue.
4. **Algorithmic core module.** Two TypeScript modules with narrow interfaces and no dependency on controllers or the ORM:
    - `SearchEngine.search(query, k)` returns matches from an in-memory flat trie. The trie is built from a snapshot of the lexicon read from PostgreSQL. When the catalog changes, a new trie is built in the background (about 235 ms at 50,000 tokens) and swapped in atomically, so searches in flight finish on the old trie.
    - `HoldQueue.enqueue`, `dispatch` and `cancel` wrap the SQL statements of Algorithm 2 in one repository module.

A typical search takes under 1 ms in the module at 50,000 tokens, so it can run on the main thread. If profiling shows event-loop delay under load, the module can move to a worker thread without changing its interface. The previous design used a separate Python and FastAPI service, a gRPC link and Redis for locking and queue persistence. Those are no longer needed, because the search index lives in the API process and PostgreSQL row locking replaces the Redis mutex.

The REST endpoints and JSON schemas in Section 4.2 of the first submission are unchanged.

Task 5: Agile Implementation Backlog (Revised Section 5.2)

5.2 Engineering Implementation and Benchmarking Tickets

Ticket 1: DEV-ALGO-101

Field	Specification
Module	`src/search/flatTrie.ts` (Node.js API)
Algorithm	Bounded Levenshtein Trie Search (flat representation)
Defenses	Reject empty queries with 400 Bad Request. Truncate or reject queries longer than 64 characters. Strip control and non-printable characters. Normalize to NFKC and lowercase before searching. Use parameterized SQL for any lexicon read.

Field	Specification
Implementation steps	1. Read lexicon tokens from PostgreSQL in batches. 2. Build the three typed arrays and the word table in breadth-first order. 3. Implement `search` with the explicit stack and preallocated DP rows. 4. Add the pruning check `min(row) > k`. 5. Add composite ranking, `Score = (1 - d / max(m, len)) * Popularity * Recency`. 6. Add atomic rebuild and swap when the catalog changes. 7. Expose `POST /api/v1/algorithm/search/fuzzy` through the Express router.
Automated tests	Results equal a brute-force Levenshtein reference on a seeded catalog for k = 0, 1 and 2. `search("algoritm", 2)` returns "algorithm". The benchmark script reports p95 latency below 35 ms at 50,000 tokens.
Definition of done	Unit tests pass with coverage above 95%. Zero mismatches against the reference. p95 below 35 ms at 50,000 tokens. Documentation updated.

Ticket 2: DEV-ALGO-102

Field	Specification
Module	`src/holds/holdQueue.ts` and a database migration (Node.js API, PostgreSQL)
Algorithm	Role-Weighted Priority Queue Dispatcher (indexed priority query)
Defenses	Empty queue returns a `NO_ACTIVE_HOLDS` result in under 1 ms without error. Reject arrival timestamps more than 5,000 ms in the future. An unrecognized role gets the Visitor weight (lowest priority). Use parameterized statements only.
Implementation steps	1. Migration: create `holds` and the index on `(title, score, seq)`. 2. Implement `enqueue`, `dispatch` and `cancel` with the statements of Algorithm 2. 3. Implement `CalculateScore` with the role weights. 4. Run `dispatch` and the copy status update in one transaction. 5. Expose `POST /api/v1/algorithm/holds/dispatch` with RFC 7807 error responses. 6. Add a scheduled job that cancels holds unclaimed after the 48-hour window.
Automated tests	Dispatch order equals an in-memory binary heap over the same holds, including cancellations. A Faculty member arriving up to 24 hours after a Student is served first. Equal roles are served first come, first served. With 50 concurrent clients draining 2,000 holds, no hold is served twice. Dispatch takes under 10 ms at 1,000 and 5,000 holds.
Definition of done	All tests above pass. Zero duplicate serves and zero deadlocks under 50 concurrent clients. Dispatch latency under 10 ms. Documentation updated.

Statements in the First Submission to Replace

The following statements no longer hold or refer to the old design. Each is listed with the replacement.

Where	First submission says	Replace with
Task 1.1, item 1	Wildcard scans cost `O(N M |Q|)` and spike above 650 ms	`ILIKE '%term%'` is a substring match with no typo tolerance: it found 3.7% of the words within two edits (measured). Its latency (about 24 ms for a 50,000-token table) is not the main problem.
Task 2.1	Pruning eliminates 98.4% of branches	Pruning leaves 96.7% of trie nodes unvisited at k = 2 and 99.6% at k = 1 (measured)

Where	First submission says	Replace with
Task 3.1	Worst case `O(m^3 |Σ|^2)`, "strictly independent" of N	Visited nodes `O(m^3 |Σ|^3)` at k = 2; time `O(m^4 |Σ|^3)` with full rows. Bounded independently of N, but actual cost grows with N (Section 3.2).
Task 3.2	About 40 node visits for m = 12; `b_eff` of 1.84	About 8,400 node visits at m = 12, k = 2, on the synthetic catalog; about 1,000 at k = 1
Task 3.2	A heap insertion sifts up 1.6 levels	0.58 levels under the assumed role mix (in-memory reference)
Task 3.3	Trie about 18.5 MB; 0.35 times N times L_avg nodes	4.4 MB at 50,000 tokens and 16.9 MB at 200,000 tokens in the flat trie; 0.545 times N times L_avg nodes (synthetic catalog)
Task 3.3	Heap under 400 KB at K = 5,000	The queue is stored in PostgreSQL; the in-memory reference heap needs about 53 KB of structure plus the hold records
Task 4.1, 6.1	Python and FastAPI engine, gRPC, Redis and Celery	TypeScript modules inside the Node.js API; PostgreSQL row locking replaces the Redis mutex; an optional worker thread replaces a separate worker service
Task 5.2	`pytest-benchmark`; Redis mutex around `ExtractMin`	The Node.js benchmark harness in `benchmarks/`; `FOR UPDATE SKIP LOCKED`
Task 6.2	98.4% of branches eliminated	96.7% (k = 2) and 99.6% (k = 1) of trie nodes unvisited
Task 6.4	Profiling with `pytest-benchmark` and `cProfile`	Per-query timing with `process.hrtime`, retained-memory measurement after forced garbage collection, and PostgreSQL round-trip timing; see Section 3.5
Algorithm names (all tasks and the poster)	Bounded Levenshtein Trie Automaton Search; Role-Weighted Composite Min-Heap Hold Dispatcher	Bounded Levenshtein Trie Search; Role-Weighted Priority Queue Dispatcher

The summary poster repeats these figures (98.4%, about 40 nodes, 18.5 MB, 400 KB, independence from N, the 650 ms baseline, FastAPI and Redis in the architecture strip, `pytest-benchmark` in the benchmarking plan) and must be updated to match.

Added References

[A1] E. Ukkonen, "Algorithms for approximate string matching," *Information and Control*, vol. 64, nos. 1-3, pp. 100-118, 1985.

[A2] W. A. Burkhard and R. M. Keller, "Some approaches to best-match file searching," *Communications of the ACM*, vol. 16, no. 4, pp. 230-236, 1973.

[A3] L. Gravano, P. G. Ipeirotis, H. V. Jagadish, N. Koudas, S. Muthukrishnan and D. Srivastava, "Approximate string joins in a database (almost) for free," in *Proc. 27th Int. Conf. on Very Large Data Bases (VLDB)*, 2001, pp. 491-500.

[A4] K. U. Schulz and S. Mihov, "Fast string correction with Levenshtein automata," *International Journal on Document Analysis and Recognition*, vol. 5, no. 1, pp. 67-85, 2002.

[A5] M. L. Fredman, R. Sedgewick, D. D. Sleator and R. E. Tarjan, "The pairing heap: A new form of self-adjusting heap," *Algorithmica*, vol. 1, no. 1, pp. 111-129, 1986.

[A6] W. Garbe, "SymSpell: Symmetric delete spelling correction algorithm," software repository, 2012. [Online]. Available: https://github.com/wolfgarbe/SymSpell

[A7] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation: SELECT, the locking clause (FOR UPDATE ... SKIP LOCKED); pg_trgm; fuzzystrmatch," 2023. [Online]. Available: https://www.postgresql.org/docs/16/

AI Assistance Disclosure

This revision was drafted with the assistance of Claude (Anthropic), which was used to write and run the prototype benchmark code, to analyze its output, and to draft the text of these sections. The team reviewed the derivations and results and is responsible for their accuracy. The statement must be adjusted to match the team's actual use and the course AI policy before submission.